

Lab 6: Provisioners

Durée: 10 minutes

Terraform peut se connecter et provisionner l'instance à distance avec un bloc d'approvisionnement.

- Tâche 1: créer un bloc de connexion à l'aide des sorties de votre module de paire de clés.
- Tâche 2: créer un bloc d'approvisionnement pour télécharger à distance du code sur votre instance.
- Tâche 3: appliquer la configuration et surveiller la connexion à distance.

Tâche 1: Créez un bloc de connexion à l'aide des sorties de votre module de paire de clés.

Étape 1.1

Dans `server/server.tf`, ajoutez un bloc de connexion dans votre ressource d'instance Web sous `tags`:

```
resource "aws_instance" "web" {  
  # Leave the first part of the block unchanged, and add a connection block:  
  # ...  
  
  connection {  
    user      = "ubuntu"  
    private_key = var.private_key  
    host      = self.public_ip  
  }  
}
```

La variable `private_key` a été définie dans le lab précédent.

La valeur de «self» fait référence à la ressource définie par le bloc courant. Donc, `self.public_ip` fait référence à l'adresse IP publique de notre `aws_instance.web`.

Tâche 2: Créez un bloc d'approvisionnement pour envoyer un code sur votre instance.

Étape 2.1

Le provisionner des fichiers s'exécutera après la création de l'instance et y copiera le contenu du répertoire `assets`. Ajoutez ceci au bloc `aws_instance` dans `server/server.tf`, juste après le bloc de connexion que vous venez d'ajouter:

```
provisioner "file" {  
  source      = "assets"  
  destination = "/tmp/"  
}
```

Étape 2.2

Le provisioner `remote-exec` exécute des commandes à distance. Nous pouvons exécuter le script à partir de notre répertoire de ressources. Ajoutez ceci après le bloc `provisioner "file"`:

```
provisioner "remote-exec" {  
  inline = [  
    "sudo sh /tmp/assets/setup-web.sh",  
  ]  
}
```

Assurez-vous que les deux approvisionnementneurs se trouvent dans le bloc de ressources `aws_instance`.

Étape 2.3

Créons maintenant notre script `setup-web.sh` sous un répertoire local `assets`:

```
#!/bin/bash  
apt-get update  
apt-get install -y apache2  
systemctl start apache2  
systemctl enable apache2  
echo "<html><h1>Welcome to Aapache Web Server</h2></html>" > /var/www/html/index.html
```

Ce script sera exécuté sur l'instance par Terraform, pour installer le serveur Apache et créer la page `index.html`

Tâche 3: appliquez votre configuration et surveillez la connexion à distance.

Étape 3.1

Un point important à retenir concernant les *provisioners* est que les ajouter ou les supprimer d'une instance ne provoquera pas la mise à jour ou la recreation de l'instance par terraform. Vous pouvez voir ceci maintenant si vous exécutez `terraform apply`:

```
...  
Apply complete! Resources: 0 added, 0 changed, 0 destroyed.  
...
```

Afin de vous assurer que les *provisioners* s'exécutent, utilisez la commande `terraform taint`. Une ressource qui a été marquée comme *tainted* sera détruite et recréée.

```
terraform taint module.server.aws_instance.web  
  
Resource instance module.server.aws_instance.web has been marked as tainted.
```

Lors de l'exécution de `terraform apply`, vous devriez voir une nouvelle sortie:

```
terraform apply
```

```
...
module.server.aws_instance.web: Provisioning with 'remote-exec'...
module.server.aws_instance.web (remote-exec): Connecting to remote host via SSH...
module.server.aws_instance.web (remote-exec): Host: 18.222.77.212
module.server.aws_instance.web (remote-exec): User: ubuntu
module.server.aws_instance.web (remote-exec): Password: false
module.server.aws_instance.web (remote-exec): Private key: true
module.server.aws_instance.web (remote-exec): SSH Agent: false
module.server.aws_instance.web (remote-exec): Connected!
module.server.aws_instance.web (remote-exec): Created symlink from /etc/systemd/system/multi-user.target.wants/webapp.service to /lib/systemd/system/webapp.service.
module.server.aws_instance.web: Creation complete after 41s (ID: i-0a869f781ab03b248)

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.
...
```

Vous pouvez maintenant visiter votre application Web en pointant votre navigateur sur la sortie `public_dns` de votre instance EC2. Si vous le souhaitez, vous pouvez également ssh sur votre instance EC2 avec une commande du type `ssh -i keys/<your_private_key>.pem ubuntu@<your_dns>`. Tapez `yes` lorsque vous êtes invité à utiliser la clé. Tapez `exit` pour quitter la session ssh.

```
ssh -i keys/<your_private_key>.pem ubuntu@<your_dns>

The authenticity of host 'ec2-54-203-220-176.us-west-2.compute.amazonaws.com (54.203.220.176)'
can't be established.
ECDSA key fingerprint is SHA256:Q/IWQRIOcb1h46ic7mHR2oBCK9XOTPHCHQ0y0A/pezs.
Are you sure you want to continue connecting (yes/no)? yes
Warning: Permanently added 'ec2-54-203-220-176.us-west-2.compute.amazonaws.com,54.203.220.176'
(ECDSA) to the list of known hosts.
Welcome to Ubuntu 16.04.6 LTS (GNU/Linux 4.4.0-1088-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

0 packages can be updated.
0 updates are security updates.

New release '18.04.2 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

Last login: Fri Aug 16 20:58:44 2020 from 34.222.21.235
ubuntu@ip-10-1-1-96:~$ exit
logout
Connection to ec2-54-203-220-176.us-west-2.compute.amazonaws.com closed.
```

Exercice

1. Créer une paire de clés SSH. `ssh-keygen -f mykey` ceci crée les deux fichiers `mykey` et `mykey.pub`
2. Ajouter des variables pour référencer les deux fichiers de la clé SSH générée, et l'utilisateur Linux par défaut dans l'instance créée

Modifier le fichiers **vars.tf** comme suit:

```
...
variable "PATH_TO_PRIVATE_KEY" {
    default = "mykey"
}
variable "PATH_TO_PUBLIC_KEY" {
    default = "mykey.pub"
}
variable "INSTANCE_USERNAME" {
    default = "ubuntu"
}
```

3. Créer une ressource de type `aws_key_pair` pour transférer la paire de clé à AWS.

Ajouter cette ressource dans le fichier **instance.tf**

```
resource "aws_key_pair" "mykey" {
    key_name     = "mykey"
    public_key   = file(var.PATH_TO_PUBLIC_KEY)
}

...
```

4. Créer un script shell pour installer Nginx dans l'instance.

```
#!/bin/bash

# sleep until instance is ready
until [[ -f /var/lib/cloud/instance/boot-finished ]]; do
    sleep 1
done

# install nginx
apt-get update
apt-get -y install nginx

# make sure nginx is started
service nginx start
```

5. Ajouter des provisioners pour envoyer le script shell et l'exécuter suite au lancement de l'instance, considérez l'ajout de la permission d'exécution au script et son lancement avec l'utilisateur `root`.

Dans le fichier **instance.tf**, ajouter le provisioner `file`, pour déployer le script shell dans l'instance, et le provisioner `remote-exec` pour exécuter le script, dans le block de la ressource `example`:

```
...

provisioner "file" {
  source      = "script.sh"
  destination = "/tmp/script.sh"
}
provisioner "remote-exec" {
  inline = [
    "chmod +x /tmp/script.sh",
    "sudo sed -i -e 's/\r$//' /tmp/script.sh", # Remove the spurious CR characters.
    "sudo /tmp/script.sh",
  ]
}
...
```

6. Indiquer la clé à utiliser dans la ressource puis ajouter un bloc de connexion dans la ressource `example`, pour permettre l'approvisionnement.

```
...
resource "aws_instance" "example" {
  ami          = var.AMIS[var.AWS_REGION]
  instance_type = "t2.micro"
  key_name     = aws_key_pair.mykey.key_name
  ...
  connection {
    host      = coalesce(self.public_ip, self.private_ip)
    type      = "ssh"
    user      = var.INSTANCE_USERNAME
    private_key = file(var.PATH_TO_PRIVATE_KEY)
  }
}
```